

PostgreSQL 数据库内核解读系列

第5节 索引1

索引基本、B-Tree索引、Hash索引、Bitmap索引、GiST索引、GIN索引、TSearch2全文搜索

于巍（花名漠雪）

阿里云数据库开源社区Maintainer

目录

CONTENT

索引基本

B-Tree索引

Hash索引

Bitmap、GIN、GIST

第1节 PostgreSQL系统概述 - 概述，架构，编译安装，主要命令

第2节 PostgreSQL的体系结构 - 系统表，启动流程，查询流程，辅助进程，Vacuum

第3节 存储管理1 - 主流存储引擎架构和原理、PG页面和元组、Buffer管理、淘汰算法

第4节 存储管理2 - 磁盘管理、VFD、空闲空间映射表（FSM）、可见性映射表（VM）、内存管理

第5节 索引 - 索引基本、B-Tree索引、Hash索引、Bitmap、GiST索引、GIN索引、TSearch2全文搜索

第6节 查询编译 - 查询分析、查询重写、查询规划、代价估计、遗传算法

第7节 查询执行 - 查询执行策略、数据定义语句执行、可优化语句执行、计划节点、其他子功能介绍

第8节 事务处理与并发控制1 - 事务系统简介、事务保存点和子事务、两阶段提交、PostgreSQL的并发控制

第9节 事务处理与并发控制2 - PostgreSQL中的三种锁、锁管理机制、死锁处理机制、多版本并发控制

第10节 事务处理与并发控制3 - 日志管理、SLRU缓冲池、CLOG、SUBTRANS、MULTIXACT、XLOG

第11节 数据库安全 - 认证方法、基于角色的权限管理、对象访问控制

第12节 复制 - 流复制、三副本、逻辑复制

第13节 分布式1 - 分布式事务、分区管理、分布式算法

第14节 分布式2 - PGXC

第15节 分布式3 - Citus

1、PostgreSQL索引基础


```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ] name ] ON [ ONLY ] table_name [ USING method ]  
    ( { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )  
[ INCLUDE ( column_name [, ...] ) ]  
[ WITH ( storage_parameter [= value] [, ... ] ) ]  
[ TABLESPACE tablespace_name ]  
[ WHERE predicate ]
```

- unique : 创建唯一索引
- concurrently : 在线创建索引，不会阻塞读写操作
- using method : 默认为btree
- asc /desc 排序，默认升序

1、Index Scan

```
postgres=# create table t(a integer, b text, c boolean);
CREATE TABLE
postgres=# insert into t(a,b,c)
postgres=#         select s.id, chr((32+random()*94)::integer), random() < 0.01
postgres=#         from generate_series(1,100000) as s(id)
postgres=#         order by random();
INSERT 0 100000
postgres=# create index on t(a);
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# explain (costs off) select * from t where a = 1;
               QUERY PLAN
-----
Index Scan using t_a_idx on t
  Index Cond: (a = 1)
(2 rows)
```

```
postgres=# select * from t order by a;
 a | b | c
-----+-----
 1 | W | f
 2 | J | f
 3 | ^ | f
 4 | , | f
 5 | f | f
 6 | R | f
 7 | Y | f
 8 | W | f
 9 | 2 | f
10 | , | f
11 | _ | f
12 | * | f
13 | 6 | f
14 | X | f
15 | U | f
16 | Y | f
17 | X | f
18 | U | f
```

2、Bitmap scan

```
postgres=# create index on t(b);
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# explain (costs off) select * from t where a <= 100 and b = 'a';
               QUERY PLAN
-----
Bitmap Heap Scan on t
  Recheck Cond: ((a <= 100) AND (b = 'a'::text))
    -> BitmapAnd
          -> Bitmap Index Scan on t_a_idx
                Index Cond: (a <= 100)
          -> Bitmap Index Scan on t_b_idx
                Index Cond: (b = 'a'::text)
(7 rows)
```

3、选择性/相关性

```
postgres=# select attname, correlation from pg_stats where tablename = 't';
 attname | correlation
-----+-----
 c       | 0.941776
 a       | -0.00149021
 b       | 0.536551
(3 rows)

postgres=# explain (costs off) select * from t where a <= 40000;
               QUERY PLAN
-----
Seq Scan on t
  Filter: (a <= 40000)
(2 rows)
```

“c” 列 接近1的绝对值表示高相关性，“a” 列接近于零的值则相反，表示无序分布. 相关性越高, 走索引扫描的离散块扫描代价越低.

条件的选择性越高，匹配的行数就越少，索引的效率就越好。

4、覆盖索引 (Covering indexes) - index only scan

```
postgres=# vacuum t;
VACUUM
postgres=# explain (analyze, costs off) select a from t where a < 100;
               QUERY PLAN
-----
Index Only Scan using t_a_idx on t (actual time=0.014..0.041 rows=99 loops=1)
  Index Cond: (a < 100)
  Heap Fetches: 0
  Planning Time: 0.351 ms
  Execution Time: 0.083 ms
(5 rows)
```


5、多列索引

```
postgres=# create index on t(a,b);
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# explain (costs off) select * from t where a <= 100 and b = 'a';
               QUERY PLAN
-----
Index Scan using t_a_b_idx on t
  Index Cond: ((a <= 100) AND (b = 'a'::text))
(2 rows)
```

7、 部分索引 (Partial indexes)

```
postgres=# create index on t(c);
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# select relpages from pg_class where relname='t_c_idx';
 relpages
-----
      276
(1 row)
```

```
postgres=# create index on t(c) where c;
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# select relpages from pg_class where relname='t_c_idx1';
 relpages
-----
         5
(1 row)
```

6、表达式索引

```
postgres=# create index on t(lower(b));
CREATE INDEX
postgres=# analyze t;
ANALYZE
postgres=# explain (costs off) select * from t where lower(b) = 'a';
               QUERY PLAN
-----
Bitmap Heap Scan on t
  Recheck Cond: (lower(b) = 'a'::text)
-> Bitmap Index Scan on t_lower_idx
    Index Cond: (lower(b) = 'a'::text)
(4 rows)
```

8、 排序中使用索引

```
postgres=# explain (costs off) select * from t order by a;
               QUERY PLAN
-----
Index Scan using t_a_idx on t
(1 row)
```

9、 并发构建索引

```
postgres=# create index concurrently on t(a);
CREATE INDEX
```

10、指定索引类型 (brin)

```
postgres=# CREATE INDEX t_brin ON t USING brin(a);
CREATE INDEX
```

1、查看PG默认支持的索引及对应Handler类型

```
SELECT *, obj_description(oid, 'pg_am' ) FROM pg_am ORDER BY 1;
```

```
postgres=# SELECT *, obj_description(oid, 'pg_am') FROM pg_am ORDER BY 1;
 amname | amhandler | amtype |      obj_description
-----+-----+-----+-----
 brin   | brinhandler | i      | block range index (BRIN) access method
 btree  | bthandler  | i      | b-tree index access method
 gin    | ginhandler | i      | GIN index access method
 gist   | gisthandler | i      | GiST index access method
 hash   | hashhandler | i      | hash index access method
 spgist | spghandler | i      | SP-GiST index access method
(6 rows)
```

2、查看B树索引属性

```
select a.amname, p.name, pg_indexam_has_property(a.oid,p.name) from pg_am a,
unnest(array['can_order','can_unique','can_multi_col','can_exclude']) p(name) where
a.amname = 'btree' order by a.amname;
```

```
postgres=# select a.amname, p.name, pg_indexam_has_property(a.oid,p.name)
postgres=# from pg_am a,
postgres=#      unnest(array['can_order','can_unique','can_multi_col','can_exclude']) p(name)
postgres=# where a.amname = 'btree'
postgres=# order by a.amname;
 amname |      name      | pg_indexam_has_property
-----+-----+-----
 btree  | can_order      | t
 btree  | can_unique     | t
 btree  | can_multi_col  | t
 btree  | can_exclude    | t
(4 rows)
```

3、索引扩展

在想要扩展的数据库下，执行下面语句
create extension postgis; -- PostGIS的矢量数据扩展
create extension postgis_raster; -- PostGIS的栅格数据扩展

```
select ST_SetSRID(ST_Point(-108,30.741),4326),
       ST_GeomFromText('POINT(-106.51 29.741)',4326)
```


索引选择 – 索引必要性



- 1、是否创建索引：索引需要消耗资源，占用磁盘空间，影响插入和更新速度，增加备份、恢复的成本，不是总需要创建索引。
- 2、索引类型选择：选择索引需要考虑索引创建和查询速度、索引尺寸、插入和更新开销、索引支持的类型和查询灵活性。

1、查看索引情况

```
postgres=# \di
public | brin_index | index | michael.yw | brin_example
public | btree_index | index | michael.yw | brin_example
public | t_a_b_idx   | index | michael.yw | t
public | t_a_idx     | index | michael.yw | t
public | t_a_idx1    | index | michael.yw | t
public | t_a_idx2    | index | michael.yw | t
public | t_a_idx3    | index | michael.yw | t
public | t_b_idx     | index | michael.yw | t
public | t_brin      | index | michael.yw | t
public | t_c_idx     | index | michael.yw | t
public | t_c_idx1    | index | michael.yw | t
public | t_lower_idx | index | michael.yw | t
```

2、查看索引和表的磁盘占用

```
postgres=# SELECT relname, pg_size_pretty(pg_relation_size(oid))
postgres-# FROM pg_class
postgres-# WHERE relname LIKE 't\_%' or relname = 't'
postgres-# ORDER BY relname;
t | 3544 kB
t_a_b_idx | 2208 kB
t_a_idx | 2208 kB
t_a_idx1 | 2208 kB
t_a_idx2 | 24 kB
t_a_idx3 | 24 kB
t_b_idx | 2208 kB
t_brin | 24 kB
t_c_idx | 2208 kB
t_c_idx1 | 40 kB
t_lower_idx | 2208 kB
```

3、查看pg_stat_user_tables中seq_scan是否很高

select relid, relname, seq_scan, idx_scan from pg_stat_user_tables;

```
postgres=# select relid, relname, seq_scan, idx_scan from pg_stat_user_tables;
16384 | t1 | 0 |
16410 | t2 | 0 |
24594 | t | 13 | 5
24589 | brin_example | 3 | 0
24583 | articles | 3 |
```

4、查看pg_stat_statements中SQL执行频率和执行时间

select query,calls,total_time,mean_time from pg_stat_statements where query like 'select%' order by mean_time desc limit 5;

```
postgres=# select query,calls,total_time,mean_time from pg_stat_statements where query like 'select%' order by mean_time desc limit 5;
select * from t order by a | 1 | 147.790293 | 147.790293
select * from t | 2 | 146.573886 | 73.286943
select * from pg_stat_user_tables | 2 | 21.293822 | 10.646911
select relid, relname, seq_scan, idx_scan from pg_stat_user_tables | 1 | 10.635014 | 10.635014
select relid, relname, seq_scan from pg_stat_user_tables | 2 | 21.183768 | 10.591884
```

5、查看pg_stat_user_indexes，如果索引调用次数很少，可能没必要创建

select relname,indexrelname,idx_scan, idx_tup_read, idx_tup_fetch from pg_stat_user_indexes;

```
postgres=# select relname,indexrelname,idx_scan, idx_tup_read, idx_tup_fetch from pg_stat_user_indexes;
brin_example | btree_index | 0 | 0 | 0
brin_example | brin_index | 0 | 0 | 0
t | t_a_idx | 4 | 100101 | 100001
t | t_b_idx | 0 | 0 | 0
t | t_a_b_idx | 1 | 1 | 0
t | t_lower_idx | 0 | 0 | 0
t | t_c_idx | 0 | 0 | 0
t | t_c_idx1 | 0 | 0 | 0
t | t_a_idx1 | 0 | 0 | 0
t | t_a_idx2 | 0 | 0 | 0
t | t_a_idx3 | 0 | 0 | 0
t | t_brin | 0 | 0 | 0
```

1、查看索引支持的类型

SELECT opfname FROM pg_opfamily, pg_am WHERE opfmethod = pg_am.oid AND amname = 'btree' ORDER BY 1;

```
postgres=# SELECT opfname FROM pg_opfamily, pg_am WHERE opfmethod = pg_am.oid AND amname = 'btree' ORDER BY 1;
 opfname
-----
abstime_ops
array_ops
bit_ops
bool_ops
bpchar_ops
bpchar_pattern_ops
bytea_ops
char_ops
datetime_ops
enum_ops
float_ops
integer_ops
interval_ops
jsonb_ops
macaddr8_ops
macaddr_ops
money_ops
name_ops
network_ops
numeric_ops
oid_ops
oidvector_ops
pg_lsn_ops
range_ops
record_image_ops
record_ops
retime_ops
text_ops
text_pattern_ops
tid_ops
time_ops
timetz_ops
tinterval_ops
tsquery_ops
tsvector_ops
uuid_ops
varbit_ops
(37 rows)
```

```
postgres=# SELECT opfname FROM pg_opfamily, pg_am WHERE opfmethod = pg_am.oid AND amname = 'gist' ORDER BY 1;
 opfname
-----
box_ops
circle_ops
jsonb_ops
network_ops
point_ops
poly_ops
range_ops
tsquery_ops
tsvector_ops
(9 rows)
```

2、查看索引支持的操作符

select amop.amopr::regoperator as opfamily_operator, amop.amopstrategy from pg_am am, pg_opfamily opf, pg_amop amop where opf.opfmethod = am.oid and amop.amopfamilly = opf.oid and am.amname = 'btree' and opf.opfname = 'bool_ops' order by amopstrategy;

```
postgres=# select amop.amopr::regoperator as opfamily_operator,
postgres=# amop.amopstrategy
postgres=# from pg_am am,
postgres=# pg_opfamily opf,
postgres=# pg_amop amop
postgres=# where opf.opfmethod = am.oid
postgres=# and amop.amopfamilly = opf.oid
postgres=# and am.amname = 'btree'
postgres=# and opf.opfname = 'bool_ops'
postgres=# order by amopstrategy;
 opfamily_operator | amopstrategy
-----+-----
<(boolean,boolean) | 1
<=(boolean,boolean) | 2
=(boolean,boolean) | 3
>=(boolean,boolean) | 4
>(boolean,boolean) | 5
(5 rows)
```

btree

```
postgres=# select amop.amopr::regoperator, amop.amoppurpose, amop.amopstrategy
postgres=# from pg_opclass opc, pg_opfamily opf, pg_am am, pg_amop amop
postgres=# where opc.opcname = 'point_ops'
postgres=# and opf.oid = opc.opcfamily
postgres=# and am.oid = opf.opfmethod
postgres=# and amop.amopfamilly = opc.opcfamily
postgres=# and am.amname = 'gist'
postgres=# and amop.amoplefttype = opc.opcintype;
 amopr | amoppurpose | amopstrategy
-----+-----+-----
<<(point,point) | s | 1
>>(point,point) | s | 5
~=(point,point) | s | 6
<^(point,point) | s | 10
>^(point,point) | s | 11
<->(point,point) | o | 15
<@(point,box) | s | 28
<@(point,polygon) | s | 48
<@(point,circle) | s | 68
(9 rows)
```

gist

```
postgres=# select amop.amopr::regoperator
ool_ops' order by amopstrategy;
 opfamily_operator | amopstrategy
-----+-----
=(boolean,boolean) | 1
(1 row)
```

hash

1、索引Handler结构

```
typedef struct IndexAmRoutine
{
    NodeTag    type;

    /*
     * Total number of strategies (operators) by which we can traverse/search
     * this AM.  Zero if AM does not have a fixed set of strategy assignments.
     */
    uint16     amstrategies;
    /* total number of support functions that this AM uses */
    uint16     amsupport;
    /* does AM support ORDER BY indexed column's value? */
    bool        amcanorder;
    /* does AM support ORDER BY result of an operator on indexed column? */
    bool        amcanorderbyop;
    /* does AM support backward scanning? */
    bool        amcanbackward;
    /* does AM support UNIQUE indexes? */
    bool        amcanunique;
    /* does AM support multi-column indexes? */
    bool        amcanmulticol;
    /* does AM require scans to have a constraint on the first index column? */
    bool        amoptionalkey;
    /* does AM handle ScalarArrayOpExpr quals? */
    bool        amsearcharray;
    /* does AM handle IS NULL/IS NOT NULL quals? */
    bool        amsearchnulls;
    /* can index storage data type differ from column data type? */
    bool        amstorage;
    /* can an index of this type be clustered on? */
    bool        amclusterable;
    /* does AM handle predicate locks? */
    bool        ampredlocks;
    /* does AM support parallel scan? */
    bool        amcanparallel;
    /* does AM support columns included with clause INCLUDE? */
    bool        amcaninclude;
    /* type of data stored in index, or InvalidOid if variable */
    Oid         amkeytype;

    /*
     * If you add new properties to either the above or the below lists, then
     * they should also (usually) be exposed via the property API (see
     * IndexAMProperty at the top of the file, and utils/adt/amutils.c).
     */

    /* interface functions */
    ambuild_function ambuild;
    ambuildempty_function ambuildempty;
    aminsert_function aminsert;
    ambulkdelete_function ambulkdelete;
    amvacuumcleanup_function amvacuumcleanup;
    amcanreturn_function amcanreturn; /* can be NULL */
    amcostestimate_function amcostestimate;
    amoptions_function amoptions;
    amproperty_function amproperty; /* can be NULL */
    amvalidate_function amvalidate;
    ambeginscan_function ambeginscan;
    amrescan_function amrescan;
    amgettuple_function amgettuple; /* can be NULL */
    amgetbitmap_function amgetbitmap; /* can be NULL */
    amendscan_function amendscan;
    ammarkpos_function ammarkpos; /* can be NULL */
    amrestrpos_function amrestrpos; /* can be NULL */

    /* interface functions to support parallel index scans */
    aestimateparallelscan_function aestimateparallelscan; /* can be NULL */
    aminitparallelscan_function aminitparallelscan; /* can be NULL */
    amparallelrescan_function amparallelrescan; /* can be NULL */
} « end IndexAmRoutine » IndexAmRoutine;
```

2、每种索引创建对应的handler，定义索引处理函数和属性

```
Datum
bthandler(PG_FUNCTION_ARGS)
{
    IndexAmRoutine *amroutine = makeNode(IndexAmRoutine);

    amroutine->amstrategies = BTMaxStrategyNumber;
    amroutine->amsupport = BTNProcs;
    amroutine->amcanorder = true;
    amroutine->amcanorderbyop = false;
    amroutine->amcanbackward = true;
    amroutine->amcanunique = true;
    amroutine->amcanmulticol = true;
    amroutine->amoptionalkey = true;
    amroutine->amsearcharray = true;
    amroutine->amsearchnulls = true;
    amroutine->amstorage = false;
    amroutine->amclusterable = true;
    amroutine->ampredlocks = true;
    amroutine->amcanparallel = true;
    amroutine->amcaninclude = true;
    amroutine->amkeytype = InvalidOid;

    amroutine->ambuild = btbuild;
    amroutine->ambuildempty = btbuildempty;
    amroutine->aminert = btinsert;
    amroutine->ambulkdelete = btbulkdelete;
    amroutine->amvacuumcleanup = btvacuumcleanup;
    amroutine->amcanreturn = btcanreturn;
    amroutine->amcostestimate = btcostestimate;
    amroutine->amoptions = btoptions;
    amroutine->amproperty = btproperty;
    amroutine->amvalidate = btvalidate;
    amroutine->ambeginscan = btbeginscan;
    amroutine->amrescan = btrescan;
    amroutine->amgettuple = btgettuple;
    amroutine->amgetbitmap = btgetbitmap;
    amroutine->amendscan = btendscan;
    amroutine->ammarkpos = btmarkpos;
    amroutine->amrestrpos = btrestpos;
    amroutine->aestimateparallelscan = btestimateparallelscan;
    amroutine->aminitparallelscan = btinitparallelscan;
    amroutine->amparallelrescan = btparallelrescan;

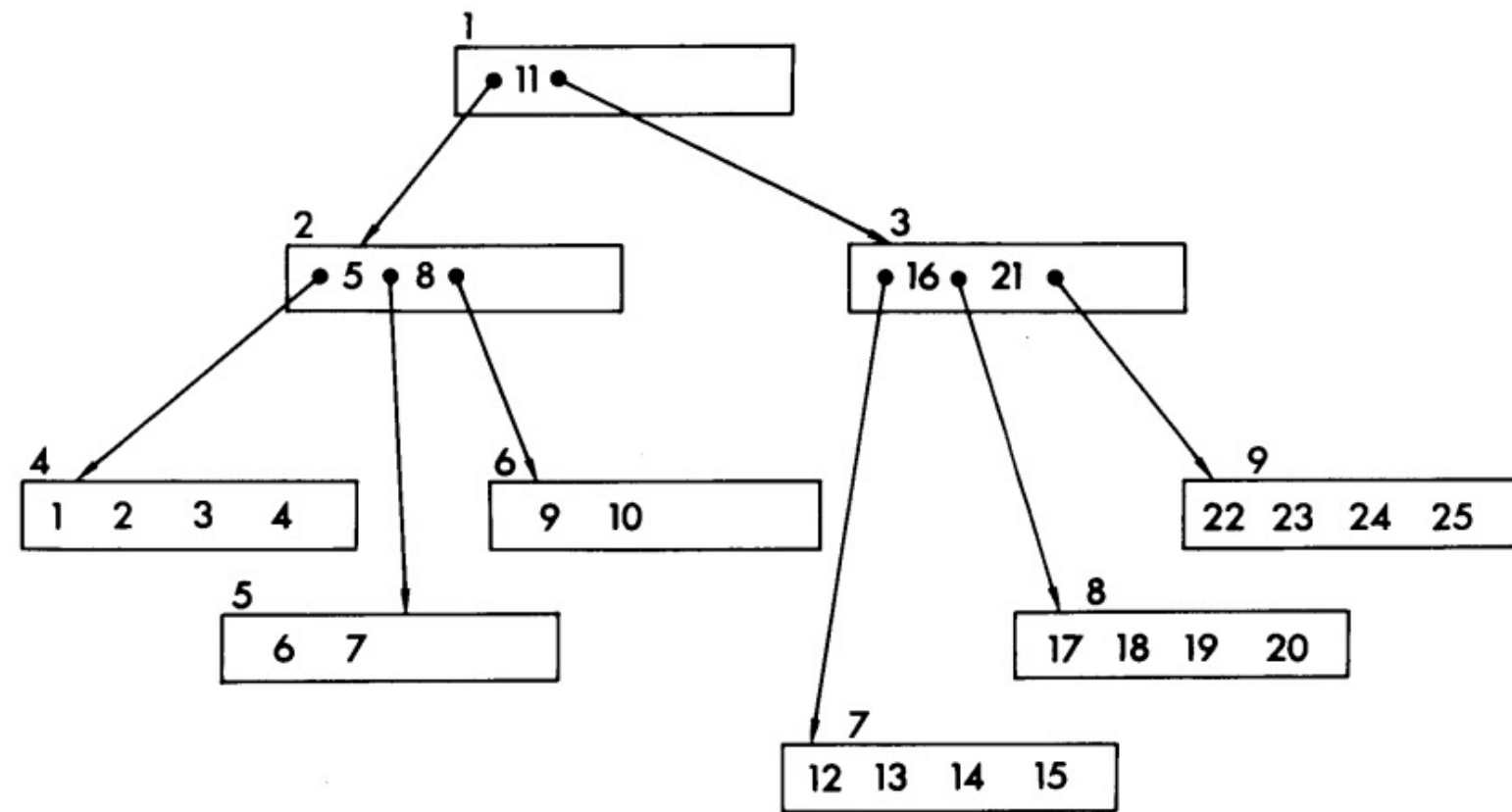
    PG_RETURN_POINTER(amroutine);
} « end bthandler »
```

3、索引对应的函数和属性在系统初始化时，创建到pg_am、pg_opfamily等系统表中

```
# Index access method handlers
{ oid => '330', descr => 'btree index access method handler',
  pronom => 'bthandler', provolatile => 'v', prorettype => 'index_am_handler',
  proargtypes => 'internal', prosrc => 'bthandler' },
{ oid => '331', descr => 'hash index access method handler',
  pronom => 'hashhandler', provolatile => 'v',
  prorettype => 'index_am_handler', proargtypes => 'internal',
  prosrc => 'hashhandler' },
```


2、B-Tree索引

B-Tree索引及原理



B-tree of order m is a tree which satisfies the following properties

1. Every node has at most m children.
2. Every internal node has at least $\lceil m/2 \rceil$ children.
3. Every non-leaf node has at least two children.
4. All leaves appear on the same level and carry no information.
5. A non-leaf node with k children contains $k-1$ keys.

Internal nodes : Internal nodes (also known as inner nodes) are all nodes except for leaf nodes and the root node.

The root node : The root node's number of children has the same upper limit as internal nodes

Leaf nodes : In Knuth's terminology, leaf nodes do not carry any information.

ORGANIZATION AND MAINTENANCE OF LARGE ORDERED INDICES

https://infolab.usc.edu/csci585/Spring2010/den_ar/indexing.pdf

Btree Indexing

=====

This directory contains a correct implementation of Lehman and Yao's high-concurrency B-tree management algorithm (P. Lehman and S. Yao, Efficient Locking for Concurrent Operations on B-Trees, ACM Transactions on Database Systems, Vol 6, No. 4, December 1981, pp 650-670). We also use a simplified version of the deletion logic described in Lanin and Shasha (V. Lanin and D. Shasha, A Symmetric Concurrent B-Tree Algorithm, Proceedings of 1986 Fall Joint Computer Conference, pp 380-389).

The basic Lehman & Yao Algorithm

Compared to a classic B-tree, L&Y adds a right-link pointer to each page, to the page's right sibling. It also adds a "high key" to each page, which is an upper bound on the keys that are allowed on that page. These two additions make it possible detect a concurrent page split, which allows the tree to be searched without holding any read locks (except to keep a single page from being modified while reading it).

When a search follows a downlink to a child page, it compares the page's high key with the search key. If the search key is greater than the high key, the page must've been split concurrently, and you must follow the right-link to find the new page containing the key range you're looking for. This might need to be repeated, if the page has been split more than once.

内存数据库更多使用 T-Tree

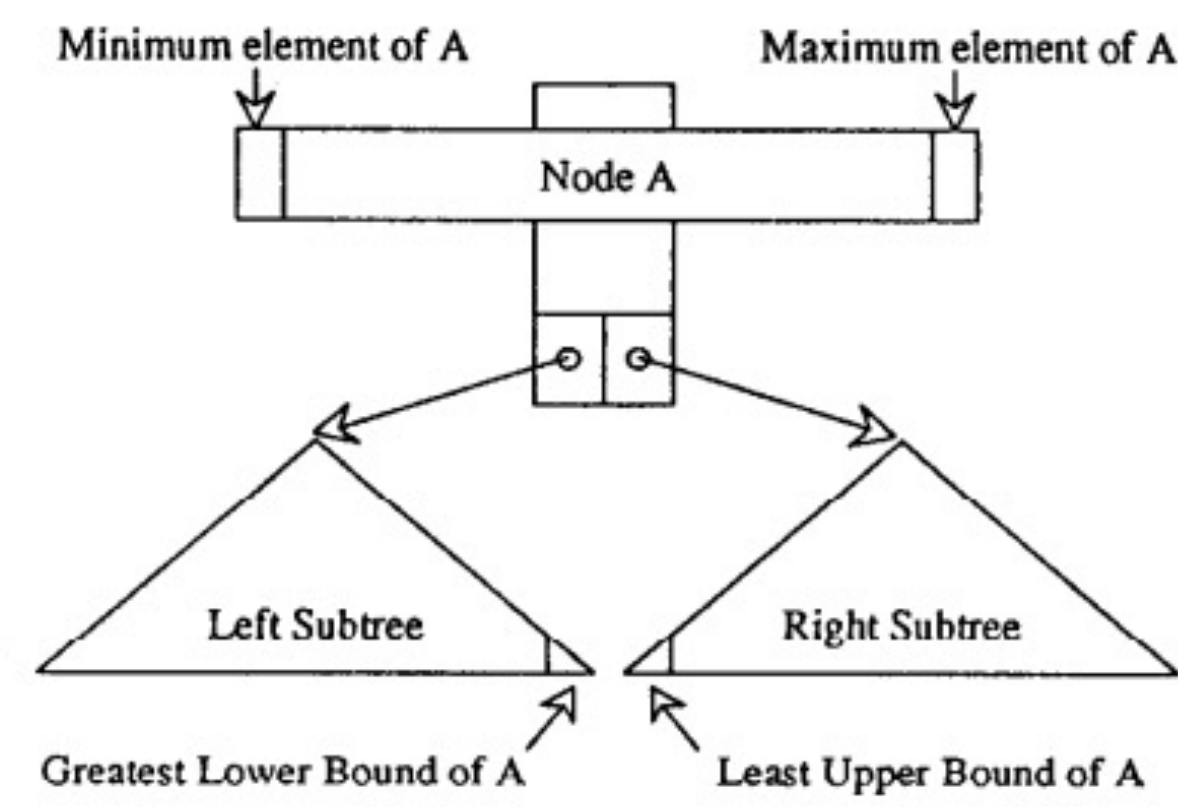
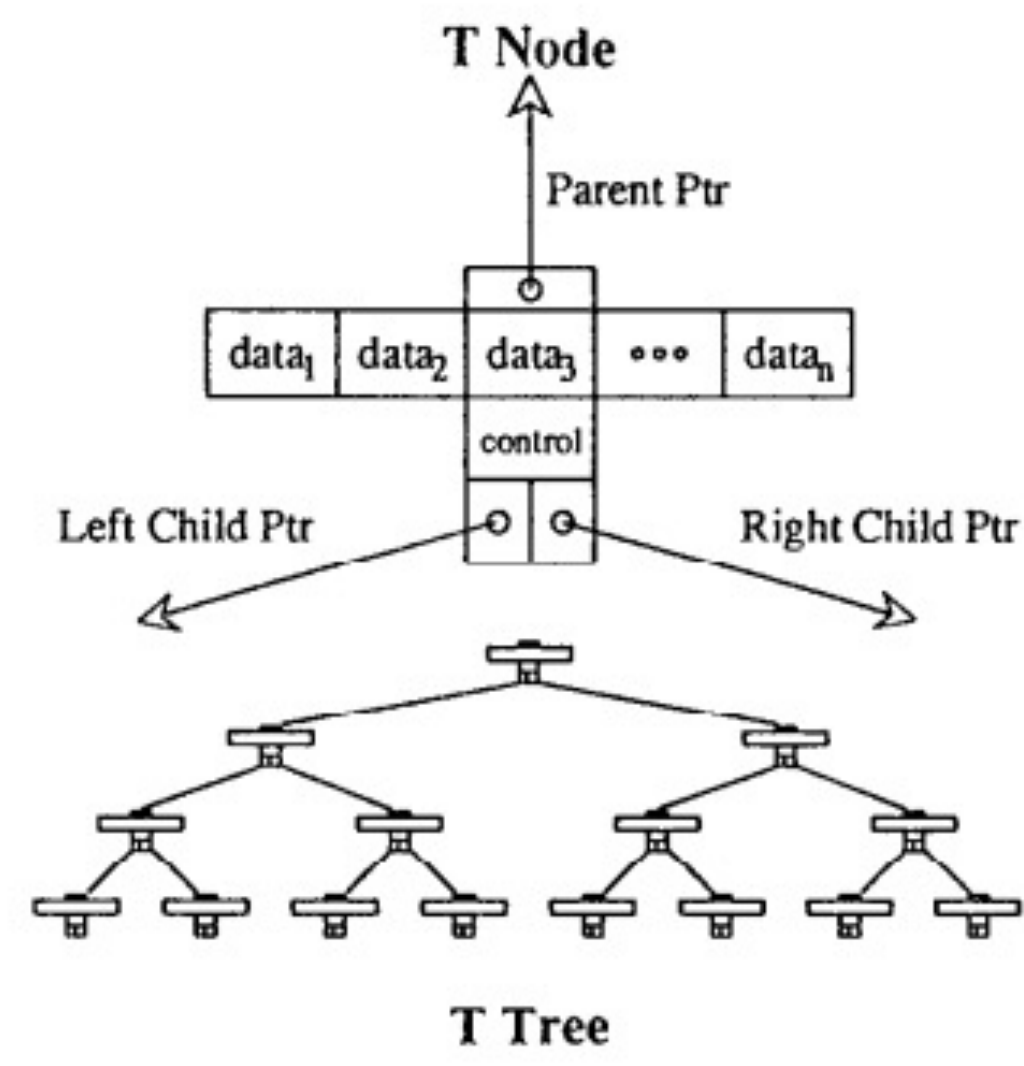


Figure 4 — The Bounds of a T-node

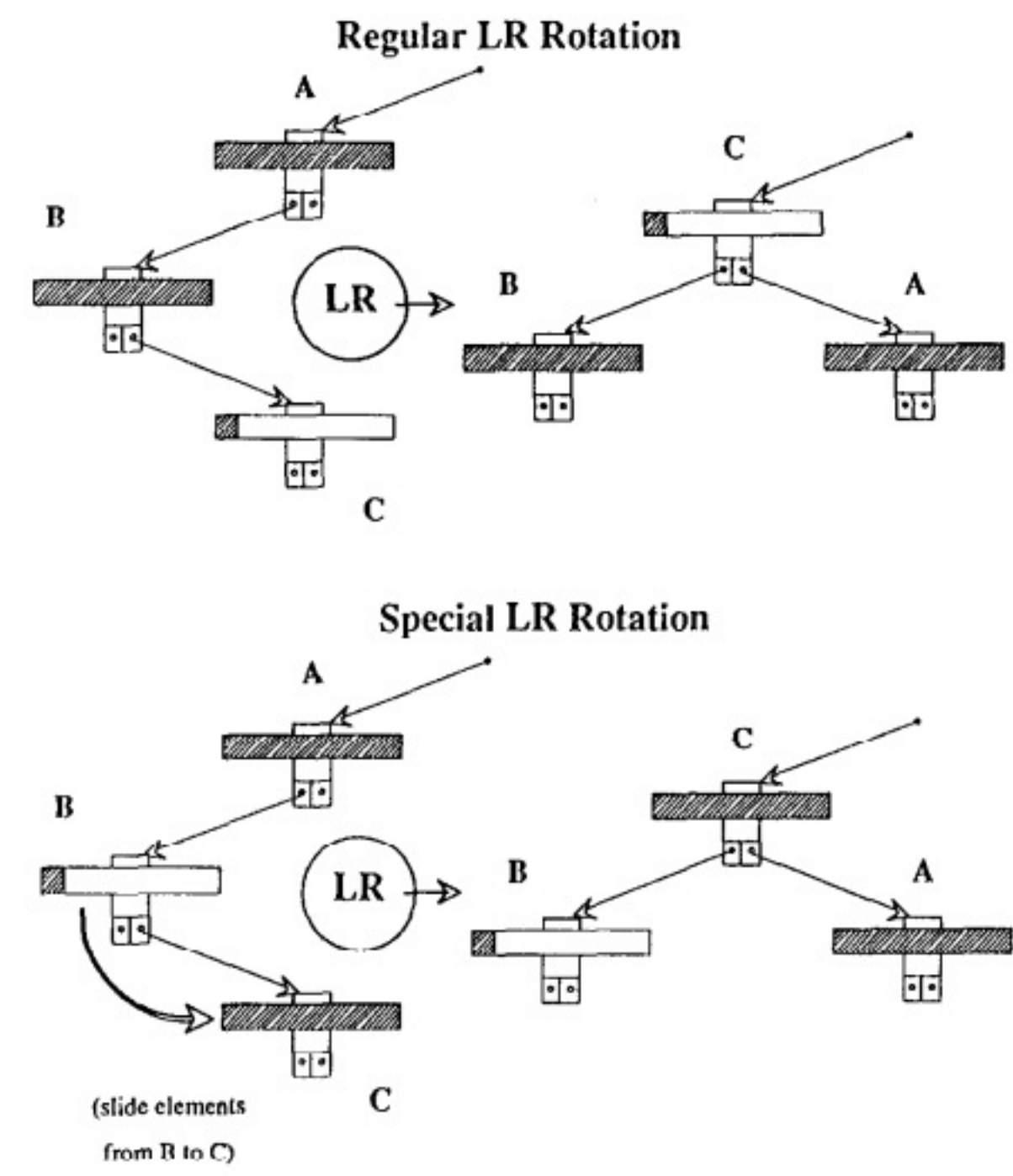
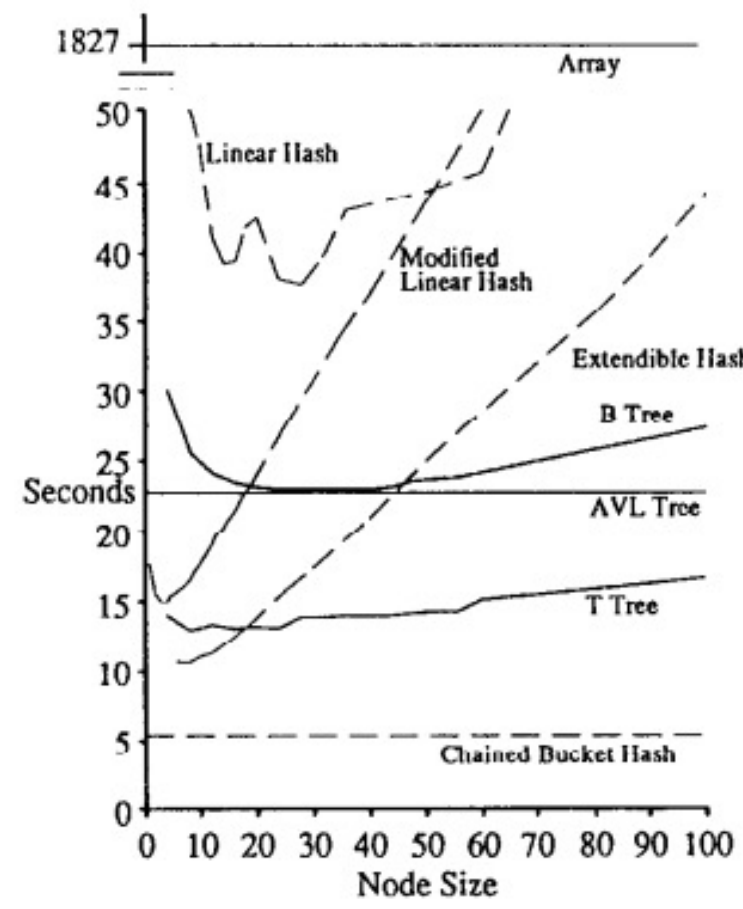
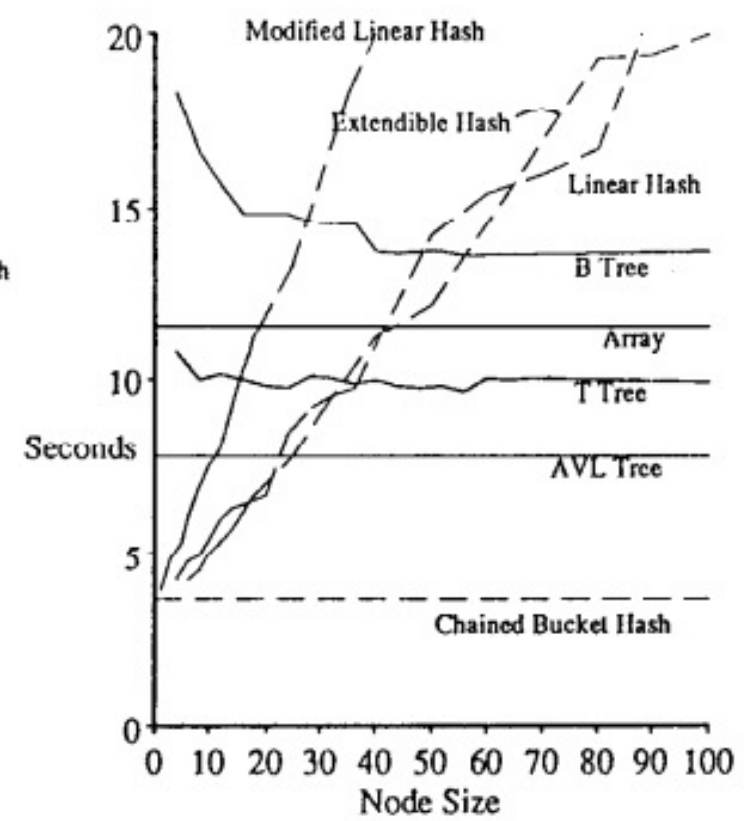


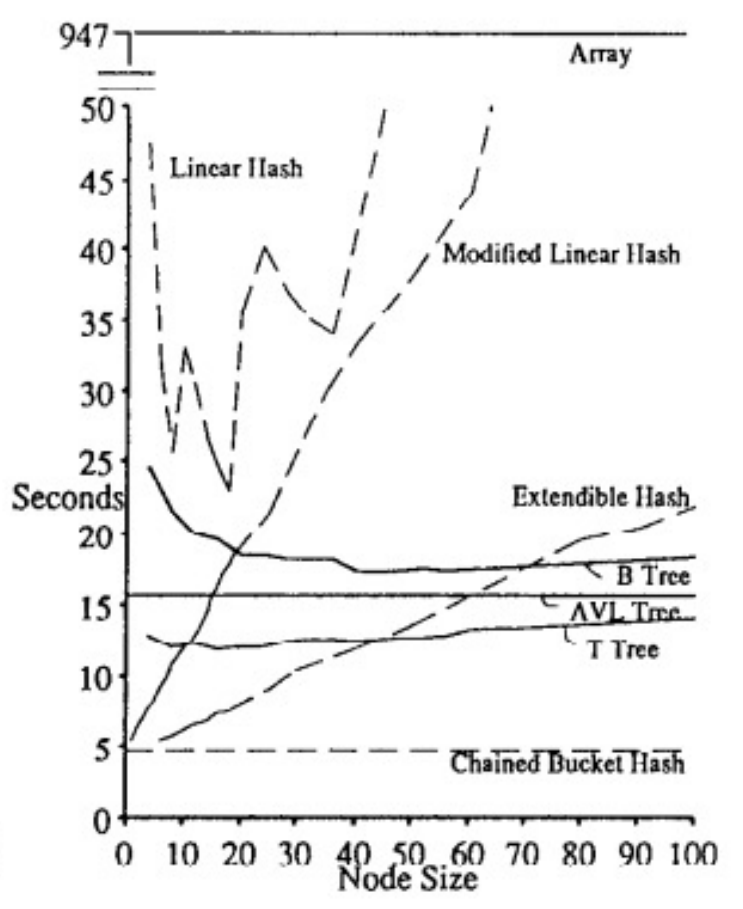
Figure 6 — Special T Tree Rebalancing Operations



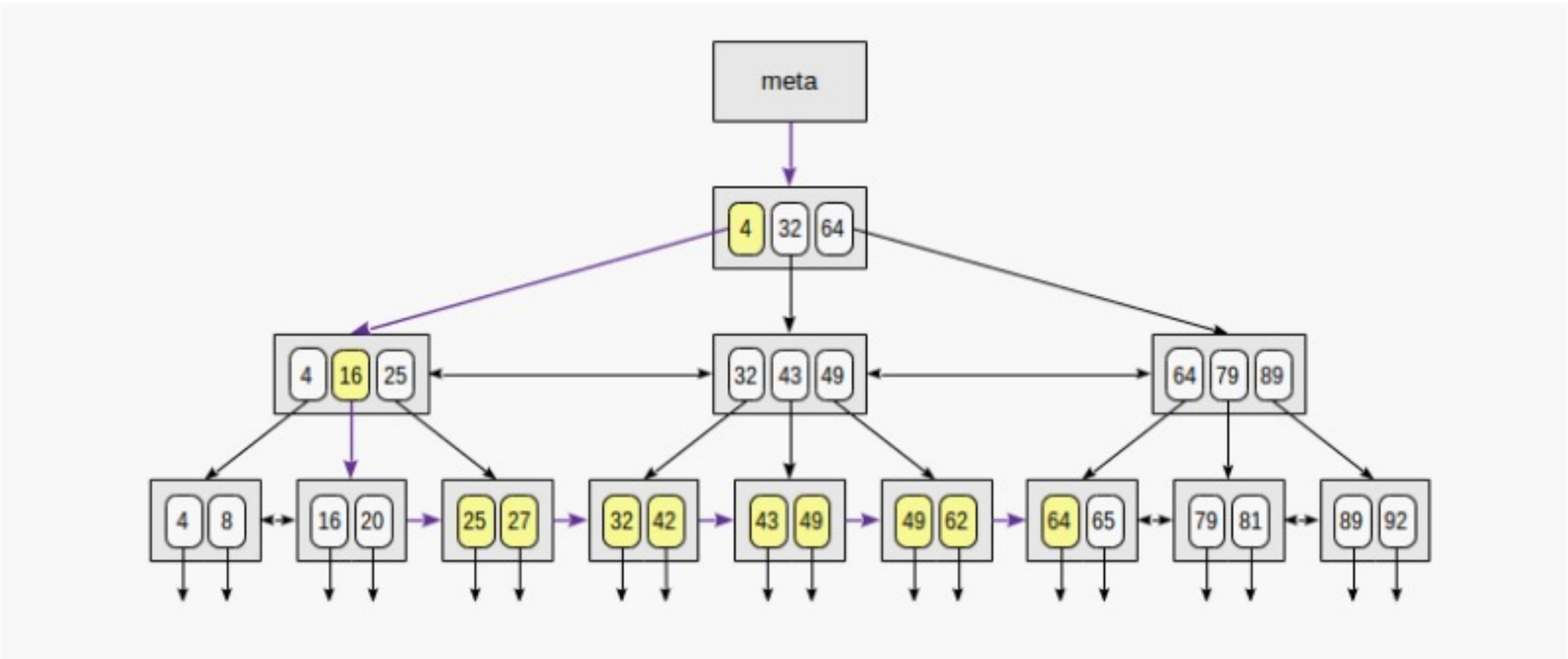
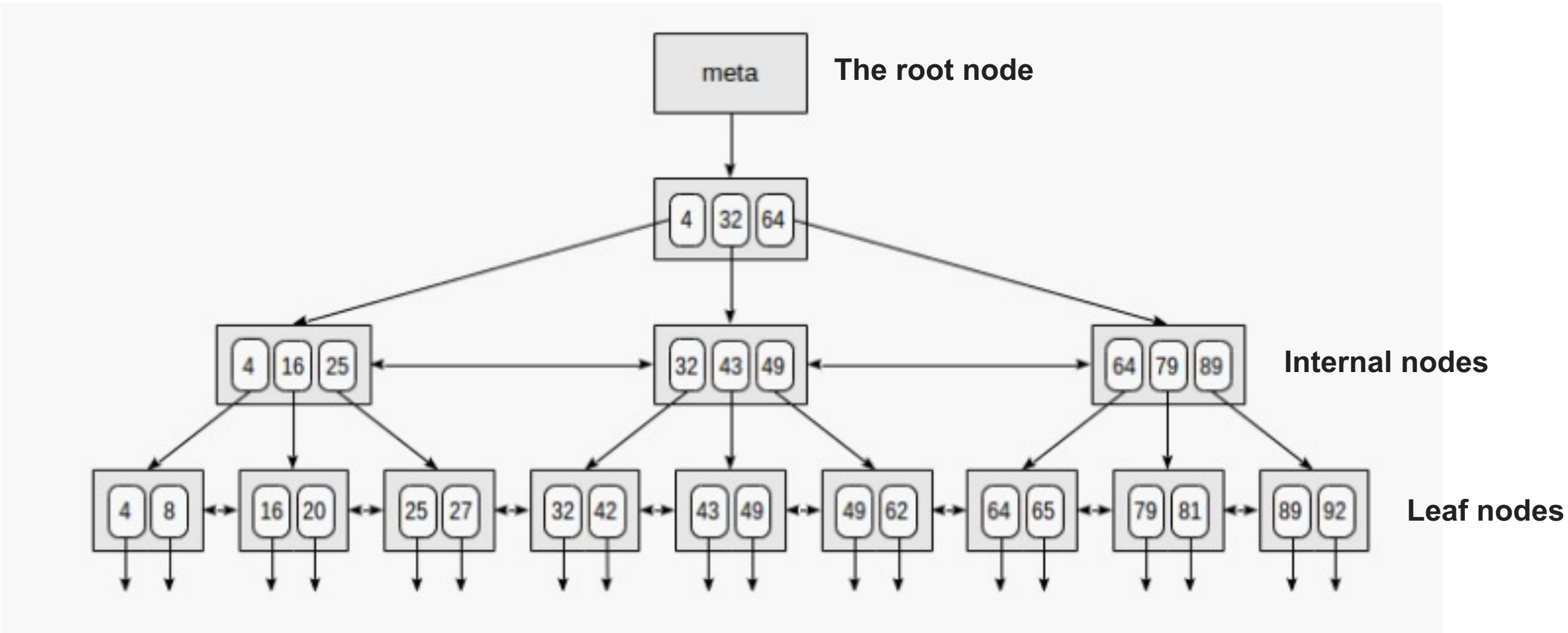
Graph 1 - Index Insertion



Graph 2 - Index Search



Graph 3 - Query Mix of 60% Searches



范围查询 23<=id<64:

B树具有一些重要的特征：

- 1. B树是平衡的，有序的, 每个叶页面与根都由相同数量的内部页面分隔开，搜索任何值都需要花费类似的时间
- 2. B树是多分支的，即每个页面包含许多TIDs。B树的深度不会太深，对于大表4-5层的深度就够了
- 3. 同一级别的页面通过双向列表相互连接。可以通过列表的一个方向或另一个方向获得有序数据集，不必每次都返回到根
- 4. B树适用于范围和等值查询，=,>,>=,<,<=,排序
- 5. 等效这些操作的其他操作例如:BETWEEN,IN以及IS NULL和字符串开头的模糊查询
- 6. 基于模式匹配时如like或~时, 仅当常量位于模式字符串的开头, 索引才会生效，例如where col like 'foo%'
- 7. 实现难度大、分裂涉及多页面修改（并行、OCC冲突检查难）、跨函数加锁、解锁

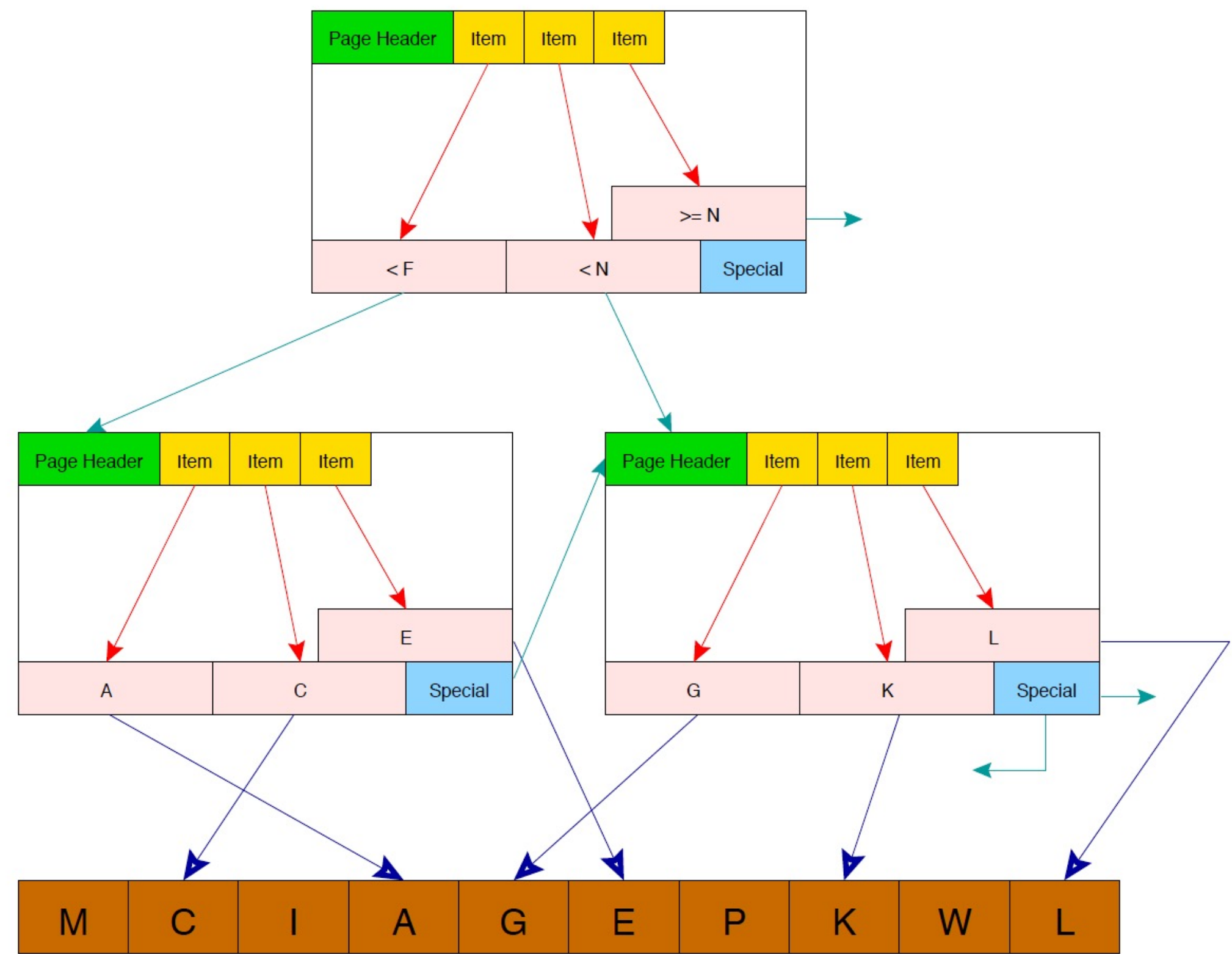
B-Tree索引页面结构



Internal

Leaf

Heap



实现说明：

- (1) 对于非叶子节点，itup指向下一个节点，而对于叶子节点，itup指向实际物理存储的位置。
- (2) Special space中，实现了两个指针，分配用于指向左右兄弟节点。
- (3) 根据BTree的特性，索引元组为有序，第一个叶子节点中itup3实际为最大索引元组，第二个叶子节点中itup1实际为最小索引元组。

1、Index Tuple结构

```
typedef struct IndexTupleData
{
    ItemPointerData t_tid; /* reference TID to heap tuple */

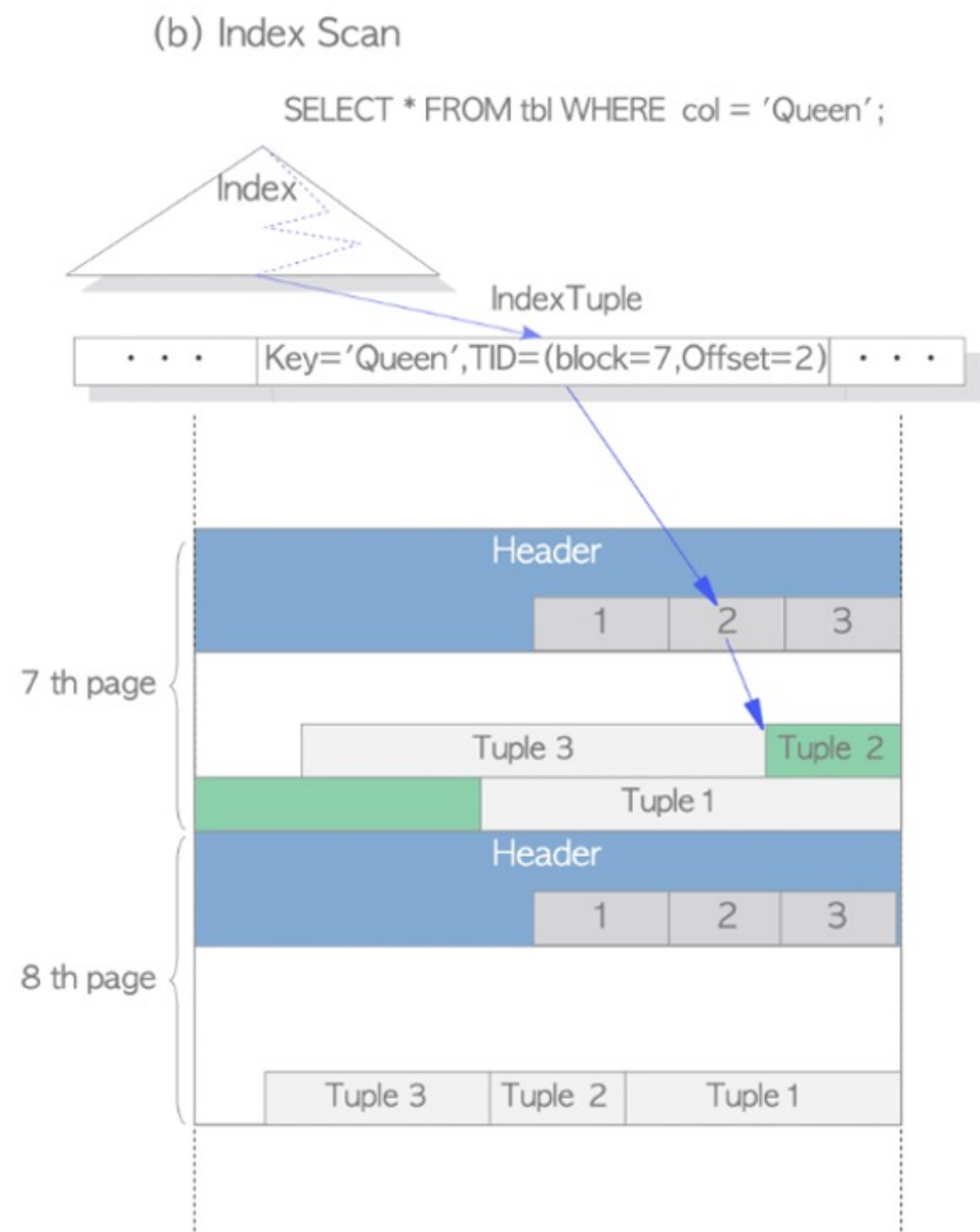
    /* -----
     * t_info is laid out in the following fashion:
     *
     * 15th (high) bit: has nulls
     * 14th bit: has var-width attributes
     * 13th bit: AM-defined meaning
     * 12-0 bit: size of tuple
     * -----
     */

    unsigned short t_info; /* various info about tuple */
} IndexTupleData; /* MORE DATA FOLLOWS AT END OF STRUCT */
```

2、指向数据指针

```
typedef struct ItemPointerData
{
    BlockIdData ip_blkid;
    OffsetNumber ip_posid;
}
```

3、索引扫描数据加载



B-Tree 常量和结构



1、Meta页面结构，记录版本，root位置、总Tuple数量等基本信息

```
/*
 * The Meta page is always the first page in the btree index.
 * Its primary purpose is to point to the location of the btree root page.
 * We also point to the "fast" root, which is the current effective root;
 * see README for discussion.
 */

typedef struct BTMetaPageData
{
    uint32      btm_magic;          /* should contain BTREE_MAGIC */
    uint32      btm_version;        /* nbtree version (always <= BTREE_VERSION) */
    BlockNumber btm_root;           /* current root location */
    uint32      btm_level;          /* tree level of the root page */
    BlockNumber btm_fastroot;       /* current "fast" root location */
    uint32      btm_fastlevel;      /* tree level of the "fast" root page */
    /* remaining fields only valid when btm_version is BTREE_VERSION */
    TransactionId btm_oldest_btpo_xact; /* oldest btpo_xact among all deleted
                                         * pages */
    float8      btm_last_cleanup_num_heap_tuples; /* number of heap tuples
                                                    * during last cleanup */
} BTMetaPageData;
```

2、Btree索引长度限制，索引字段不能超过1/3页面长度

```
/*
 * Maximum size of a btree index entry, including its tuple header.
 *
 * We actually need to be able to fit three items on every page,
 * so restrict any one item to 1/3 the per-page available space.
 */
#define BTMaxItemSize(page) \
    MAXALIGN_DOWN ( (PageGetPageSize(page) - \
                     MAXALIGN(sizeof(PageHeaderData) + 3*sizeof(ItemIdData)) - \
                     MAXALIGN(sizeof(BTPageOpaqueData))) / 3 )
```

3、默认页面填充读参数

```
/*
 * The leaf-page fillfactor defaults to 90% but is user-adjustable.
 * For pages above the leaf level, we use a fixed 70% fillfactor.
 * The fillfactor is applied during index build and when splitting
 * a rightmost page; when splitting non-rightmost pages we try to
 * divide the data equally.
 */
#define BTREE_MIN_FILLFACTOR      10
#define BTREE_DEFAULT_FILLFACTOR  90
#define BTREE_NONLEAF_FILLFACTOR 70
```

4、索引页面结尾，记录页面类型、层级、左右链接等

```
typedef struct BTPageOpaqueData
{
    BlockNumber btpo_prev;          /* left sibling, or P_NONE if leftmost */
    BlockNumber btpo_next;          /* right sibling, or P_NONE if rightmost */
    union
    {
        uint32      level;          /* tree level --- zero for leaf pages */
        TransactionId xact;          /* next transaction ID, if deleted */
    } btpo;
    uint16      btpo_flags;          /* flag bits, see below */
    BTCycleId    btpo_cycleid;       /* vacuum cycle ID of latest split */
} BTPageOpaqueData;
```

5、页面类型及状态标记

```
/* Bits defined in btpo_flags */
#define BTP_LEAF      (1 << 0) /* leaf page, i.e. not internal page */
#define BTP_ROOT      (1 << 1) /* root page (has no parent) */
#define BTP_DELETED   (1 << 2) /* page has been deleted from tree */
#define BTP_META      (1 << 3) /* meta-page */
#define BTP_HALF_DEAD (1 << 4) /* empty, but still in tree */
#define BTP_SPLIT_END (1 << 5) /* rightmost page of split group */
#define BTP_HAS_GARBAGE (1 << 6) /* page has LP_DEAD tuples */
#define BTP_INCOMPLETE_SPLIT (1 << 7) /* right sibling's downlink is missing */
```

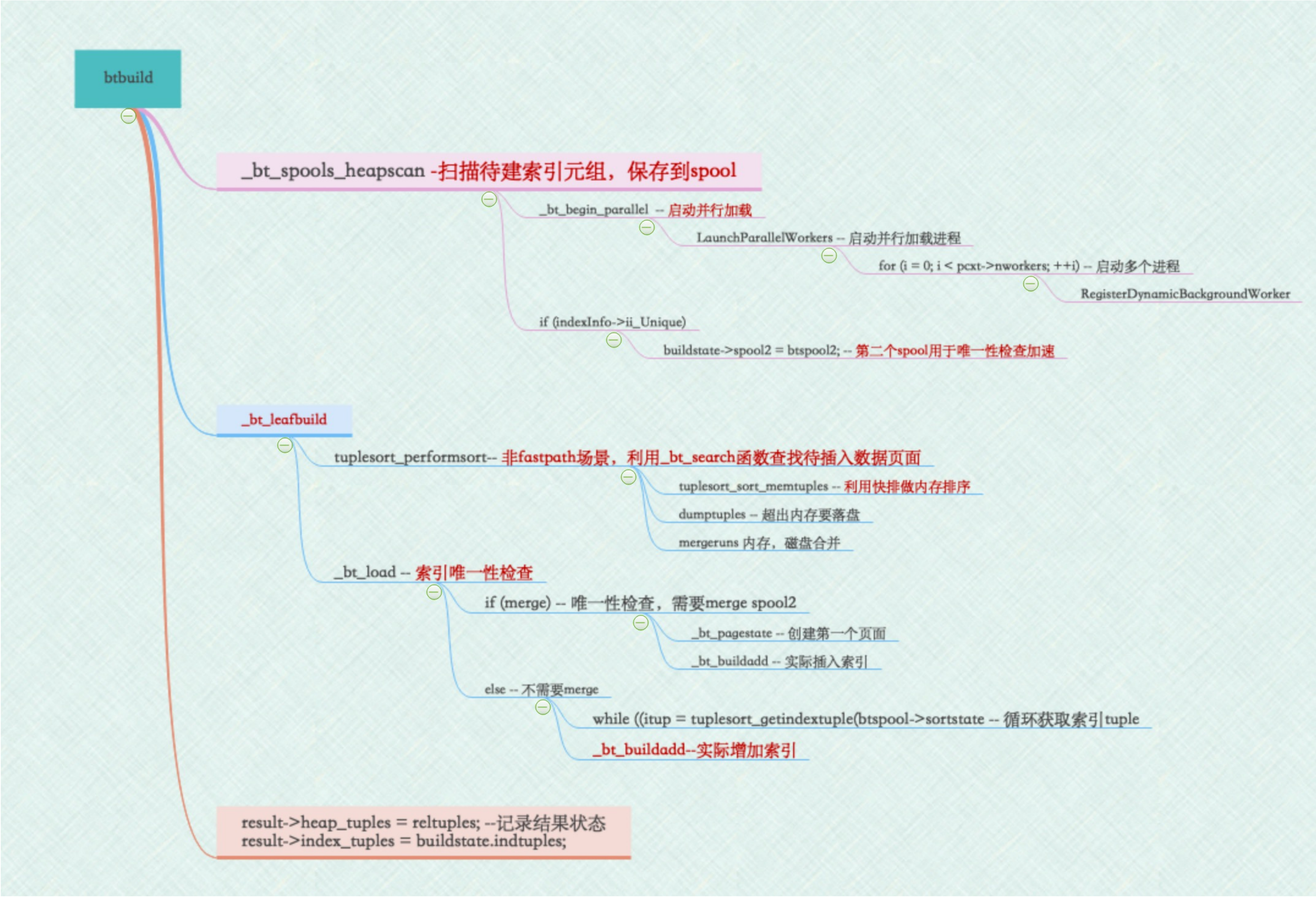
6、Magic等常量定义

```
#define BTREE_METAPAGE 0 /* first page is meta */
#define BTREE_MAGIC 0x053162 /* magic number of btree pages */
#define BTREE_VERSION 3 /* current version number */
#define BTREE_MIN_VERSION 2 /* minimal supported version number */
```

7、Hi key位置定义

```
/*
 * Lehman and Yao's algorithm requires a ``high key'' on every non-rightmost
 * page. The high key is not a data key, but gives info about what range of
 * keys is supposed to be on this page. The high key on a page is required
 * to be greater than or equal to any data key that appears on the page.
 * If we find ourselves trying to insert a key > high key, we know we need
 * to move right (this should only happen if the page was split since we
 * examined the parent page).
 *
 * Our insertion algorithm guarantees that we can use the initial least key
 * on our right sibling as the high key. Once a page is created, its high
 * key changes only if the page is split.
 *
 * On a non-rightmost page, the high key lives in item 1 and data items
 * start in item 2. Rightmost pages have no high key, so we store data
 * items beginning in item 1.
 */
#define P_HIKEY ((OffsetNumber) 1)
#define P_FIRSTKEY ((OffsetNumber) 2)
#define P_FIRSTDATAKEY(opaque) (P_RIGHTMOST(opaque) ? P_HIKEY : P_FIRSTKEY)
```


B-Tree索引构建流程



```
Datum
bthandler(PG_FUNCTION_ARGS)
{
    IndexAmRoutine *amroutine = makeNode(IndexAmRoutine);

    amroutine->amstrategies = BTMaxStrategyNumber;
    amroutine->amsupport = BTNProcs;
    amroutine->amcanorder = true;
    amroutine->amcanorderbyop = false;
    amroutine->amcanbackward = true;
    amroutine->amcanunique = true;
    amroutine->amcanmulticol = true;
    amroutine->amoptionalkey = true;
    amroutine->amsearcharray = true;
    amroutine->amsearchnulls = true;
    amroutine->amstorage = false;
    amroutine->amclusterable = true;
    amroutine->ampredlocks = true;
    amroutine->amcanparallel = true;
    amroutine->amcaninclude = true;
    amroutine->amkeytype = InvalidOid;

    amroutine->ambuild = btbuild;
    amroutine->ambuildempty = btbuildempty;
    amroutine->aminert = btinsert;
    amroutine->ambulkdelete = btbulkdelete;
    amroutine->amvacuumcleanup = btvacuumcleanup;
    amroutine->amcanreturn = btcanreturn;
    amroutine->amcostestimate = btcostestimate;
    amroutine->amoptions = btoptions;
    amroutine->amproperty = btproperty;
    amroutine->amvalidate = btvalidate;
    amroutine->ambeginscan = btbeginscan;
    amroutine->amrescan = btrescan;
    amroutine->amgettupple = btgettupple;
    amroutine->amgetbitmap = btgetbitmap;
    amroutine->amendscan = btendscan;
    amroutine->ammarkpos = btmarkpos;
    amroutine->amrestrpos = btrestpos;
    amroutine->amestimateparallelscan = btestimateparallelscan;
    amroutine->aminitparallelscan = btinitparallelscan;
    amroutine->amparallelrescan = btparallelrescan;

    PG_RETURN_POINTER(amroutine);
} << end bthandler >>
```


Btree spool 及排序



1、排序状态跟踪

```
/* Private state of a Tuplesort operation.
 */
struct Tuplesortstate
{
    TupSortStatus status; /* enumerated value as shown above */
    int nKeys; /* number of columns in sort key */
    bool randomAccess; /* did caller request random access? */
    bool bounded; /* did caller specify a maximum number of
                  * tuples to return? */
    bool boundUsed; /* true if we made use of a bounded heap */
    int bound; /* if bounded, the maximum number of tuples */
    bool tuples; /* Can SortTuple.tuple ever be set? */
    int64 availMem; /* remaining memory available, in bytes */
    int64 allowedMem; /* total memory allowed, in bytes */
    int maxTapes; /* number of tapes (Knuth's T) */
    int tapeRange; /* maxTapes-1 (Knuth's P) */
    MemoryContext sortcontext; /* memory context holding most sort data */
    MemoryContext tuplecontext; /* sub-context of sortcontext for tuple data */
    LogicalTapeSet *tapeset; /* logtape.c object for tapes in a temp file */

    /*
     * These function pointers decouple the routines that must know what kind
     * of tuple we are sorting from the routines that don't need to know it.
     * They are set up by the tuplesort_begin_xxx routines.
     *
     * Function to compare two tuples; result is per qsort() convention, ie:
     * <0, 0, >0 according as a<b, a=b, a>b. The API must match
     * qsort_arg_comparator.
     */
    SortTupleComparator comparetup;

    /*
     * Function to copy a supplied input tuple into palloc'd space and set up
     * its SortTuple representation (ie, set tuple/datum1/isnull1). Also,
     * state->availMem must be decreased by the amount of space used for the
     * tuple copy (note the SortTuple struct itself is not counted).
     */
    void (*copytup) (Tuplesortstate *state, SortTuple *stup, void *tup);

    /*
     * Function to write a stored tuple onto tape. The representation of the
     * tuple on tape need not be the same as it is in memory; requirements on
     * the tape representation are given below. Unless the slab allocator is
     * used, after writing the tuple, pfree() the out-of-line data (not the
     * SortTuple struct!), and increase state->availMem by the amount of
     * memory space thereby released.
     */
    void (*writetup) (Tuplesortstate *state, int tapenum,
                     SortTuple *stup);

    /*
     * Function to read a stored tuple from tape back into memory. 'len' is
     * the already-read length of the stored tuple. The tuple is allocated
     * from the slab memory arena, or is palloc'd, see readtup_alloc().
     */
    void (*readtup) (Tuplesortstate *state, SortTuple *stup,
                    int tapenum, unsigned int len);

    /*
     * This array holds the tuples now in sort memory. If we are in state
     * INITIAL, the tuples are in no particular order; if we are in state
     * SORTEDINMEM, the tuples are in final sorted order; in states BUILDRUNS
     * and FINALMERGE, the tuples are organized in "heap" order per Algorithm
     * H. In state SORTEDONTAPE, the array is not used.
     */
    SortTuple *memtuples; /* array of SortTuple structs */
    int memtupcount; /* number of tuples currently present */
    int memtupsize; /* allocated length of memtuples array */
    bool growmemtuples; /* memtuples' growth still underway? */
};
```

2、构建索引时，用于记录排序状态

```
/*
 * Status record for spooling/sorting phase. (Note we may have two of
 * these due to the special requirements for uniqueness-checking with
 * dead tuples.)
 */
typedef struct BTSpool
{
    Tuplesortstate *sortstate; /* state data for tuplesort.c */
    Relation heap;
    Relation index;
    bool isunique;
} BTSpool;
```

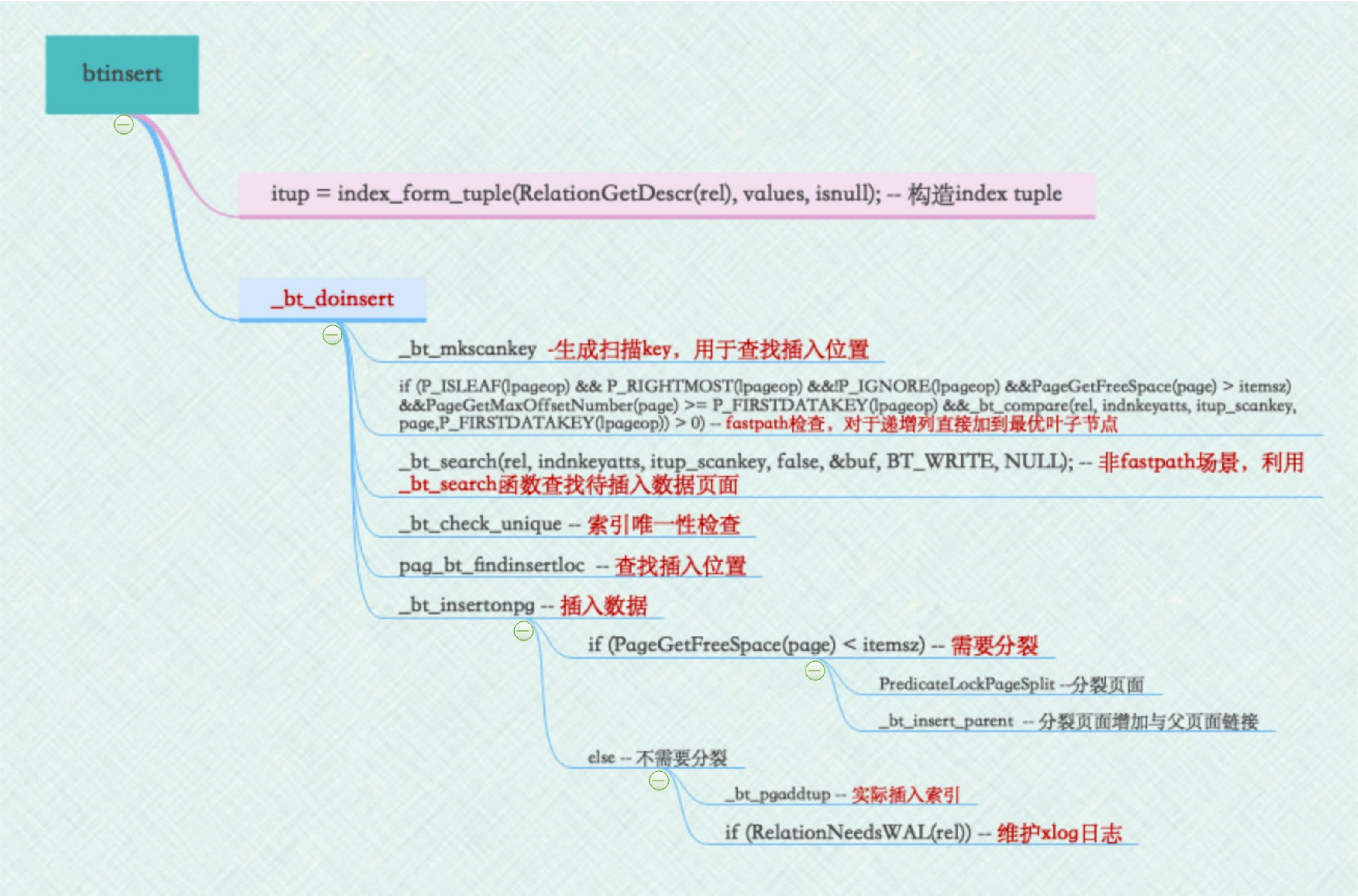
3、排序tuple

```
typedef struct
{
    void *tuple; /* the tuple itself */
    Datum datum1; /* value of first key column */
    bool isnull1; /* is first key column NULL? */
    int tupindex; /* see notes above */
} SortTuple;
```

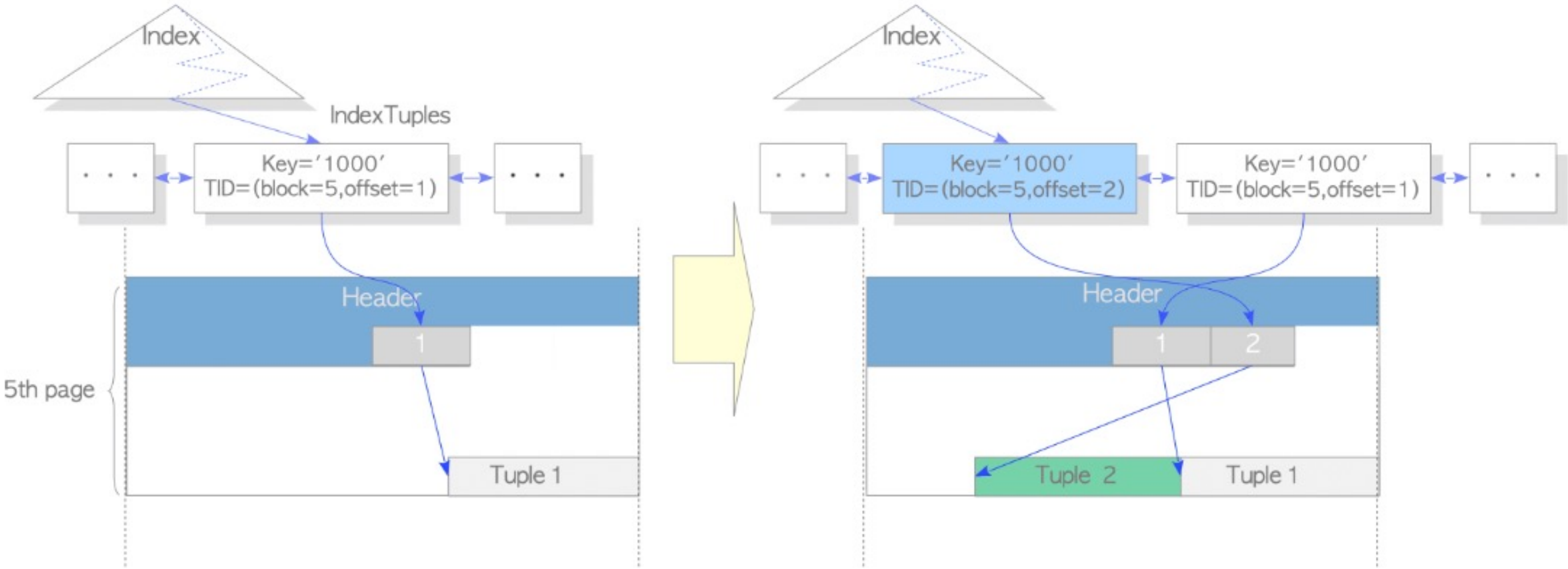
4、排序状态

```
/*
 * Possible states of a Tuplesort object. These denote the states that
 * persist between calls of Tuplesort routines.
 */
typedef enum
{
    TSS_INITIAL, /* Loading tuples; still within memory limit */
    TSS_BOUNDED, /* Loading tuples into bounded-size heap */
    TSS_BUILDRUNS, /* Loading tuples; writing to tape */
    TSS_SORTEDINMEM, /* Sort completed entirely in memory */
    TSS_SORTEDONTAPE, /* Sort completed, final run is on tape */
    TSS_FINALMERGE /* Performing final merge on-the-fly */
} TupSortStatus;
```


B-Tree索引插入流程



Heap Only Tuple (HOT) 优化



	t_xmin	t_xmax	t_ctid	t_infomask2	user data
Tuple_1	99	100	(5,1)		1000, 'A'
Tuple_1	99	100	(5,2)	HEAP_HOT_UPDATED	1000, 'A'
Tuple_2	100	0	(5,2)	HEAP_ONLY_TUPLE	1000, 'B'

1、只更改数据页面，不更改索引

```
typedef struct HeapTupleFields
{
    TransactionId t_xmin; /* inserting xact ID */
    TransactionId t_xmax; /* deleting or locking xact ID */

    union
    {
        CommandId t_cid; /* inserting or deleting command ID, or both */
        TransactionId t_xvac; /* old-style VACUUM FULL xact ID */
        t_field3;
    }
} HeapTupleFields;
```

2、Hot update 标记

```
#define HEAP_HOT_UPDATED 0x4000 /* tuple was HOT-updated */
#define HEAP_ONLY_TUPLE 0x8000 /* this is heap-only tuple */
```

3、HOT Tuple 判断

```
/*
 * Note that we stop considering a tuple HOT-updated as soon as it is known
 * aborted or the would-be updating transaction is known aborted. For best
 * efficiency, check tuple visibility before using this macro, so that the
 * INVALID bits will be as up to date as possible.
 */
#define HeapTupleHeaderIsHotUpdated(tup) \
( \
    ((tup)->t_infomask2 & HEAP_HOT_UPDATED) != 0 && \
    ((tup)->t_infomask & HEAP_XMAX_INVALID) == 0 && \
    !HeapTupleHeaderXminInvalid(tup) \
)
```

4、Buffer内查找Hot tuple

```
/*
 * heap_hot_search_buffer - search HOT chain for tuple satisfying snapshot
 *
 * On entry, *tid is the TID of a tuple (either a simple tuple, or the root
 * of a HOT chain), and buffer is the buffer holding this tuple. We search
 * for the first chain member satisfying the given snapshot. If one is
 * found, we update *tid to reference that tuple's offset number, and
 * return true. If no match, return false without modifying *tid.
 *
 * heapTuple is a caller-supplied buffer. When a match is found, we return
 * the tuple here, in addition to updating *tid. If no match is found, the
 * contents of this buffer on return are undefined.
 *
 * If all dead is not NULL, we check non-visible tuples to see if they are
 * globally dead; *all dead is set true if all members of the HOT chain
 * are vacuumable, false if not.
 *
 * Unlike heap fetch, the caller must already have pin and (at least) share
 * lock on the buffer; it is still pinned/locked at exit. Also unlike
 * heap_fetch, we do not report any pgstats count; caller may do so if wanted.
 */
bool
heap_hot_search_buffer(ItemPointer tid, Relation relation, Buffer buffer,
                      Snapshot snapshot, HeapTuple heapTuple,
                      bool *all_dead, bool first_call)
```

5、判断和查找HOT Tuple

```
/*
 * Check to see if HOT chain continues past this tuple; if so fetch
 * the next offnum and loop around.
 */
if (HeapTupleIsHotUpdated(heapTuple))
{
    Assert(ItemPointerGetBlockNumber(&heapTuple->t_data->t_ctid) ==
           blkno);
    offnum = ItemPointerGetOffsetNumber(&heapTuple->t_data->t_ctid);
    at_chain_start = false;
    prev_xmax = HeapTupleHeaderGetUpdateXid(heapTuple->t_data);
}
```


Index-Only Scans 优化



```
testdb=# SELECT id, name FROM tbl WHERE id BETWEEN 18 and 19;
 id | name 
----+-----
 18 | Queen
 19 | Boston
(2 rows)
```

1、Index only scan算子

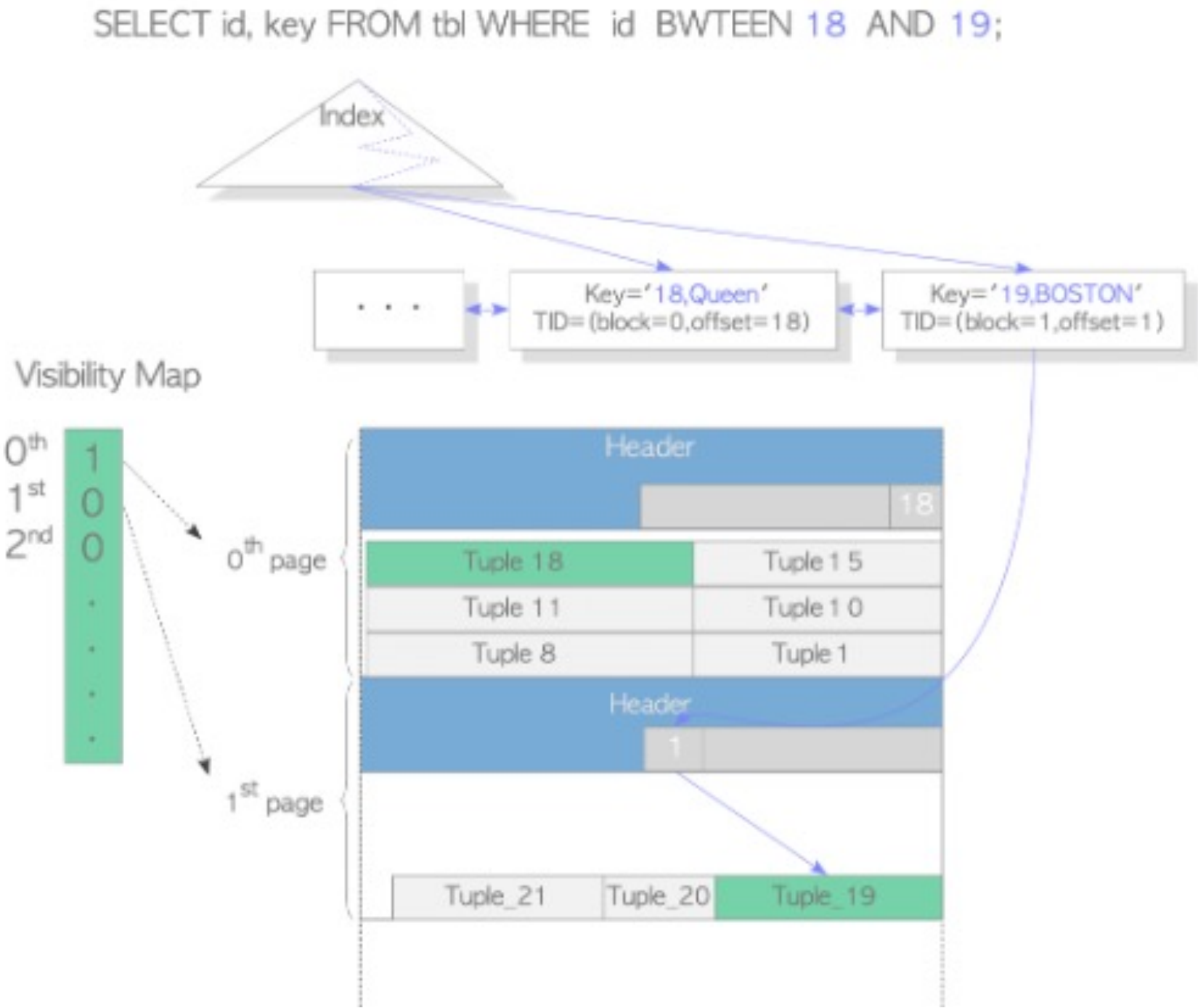
```
typedef struct IndexOnlyScan
{
    Scan      scan;
    Oid        indexid;          /* OID of index to scan */
    List       *indexqual;        /* list of index quals (usually OpExprs) */
    List       *indexorderby;    /* list of index ORDER BY exprs */
    List       *indexlist;       /* TargetEntry list describing index's cols */
    ScanDirection indexorderdir; /* forward or backward or don't care */
    List       *recheckqual;     /* index quals in recheckable form */
} IndexOnlyScan;
```

2、IndexOnlyNext函数中主要逻辑

```
/*
 * OK, now that we have what we need, fetch the next tuple.
 */
while ((tid = index_getnext_tid(scandesc, direction)) != NULL)
{
    HeapTuple tuple = NULL;

    CHECK_FOR_INTERRUPTS();

    /*
     * We can skip the heap fetch if the TID references a heap page on
     * which all tuples are known visible to everybody. In any case,
     * we'll use the index tuple not the heap tuple as the data source.
     *
     * Note on Memory Ordering Effects: visibilitymap_get_status does not
     * lock the visibility map buffer, and therefore the result we read
     * here could be slightly stale. However, it can't be stale enough to
     * matter.
     *
     * We need to detect clearing a VM bit due to an insert right away,
     * because the tuple is present in the index page but not visible. The
     * reading of the TID by this scan (using a shared lock on the index
     * buffer) is serialized with the insert of the TID into the index
     * (using an exclusive lock on the index buffer). Because the VM bit
     * is cleared before updating the index, and locking/unlocking of the
     * index page acts as a full memory barrier, we are sure to see the
     * cleared bit if we see a recently-inserted TID.
     *
     * Deletes do not update the index page (only VACUUM will clear out
     * the TID), so the clearing of the VM bit by a delete is not
     * serialized with this test below, and we may see a value that is
     * significantly stale. However, we don't care about the delete right
     * away, because the tuple is still visible until the deleting
     * transaction commits or the statement ends (if it's our
     * transaction). In either case, the lock on the VM buffer will have
     * been released (acting as a write barrier) after clearing the bit.
     * And for us to have a snapshot that includes the deleting
     * transaction (making the tuple invisible), we must have acquired
     * ProcArrayLock after that time, acting as a read barrier.
     *
     * It's worth going through this complexity to avoid needing to lock
     * the VM buffer, which could cause significant contention.
     */
    if (!VM_ALL_VISIBLE(scandesc->heapRelation,
                        ItemPointerGetBlockNumber(tid),
                        &node->ioss_VMBuffer))
    {
        /*
         * Rats, we have to visit the heap to check visibility.
         */
        InstrCountTuples2(node, 1);
        tuple = index_fetch_heap(scandesc);
        if (tuple == NULL)
            continue; /* no visible tuple, try next index entry */
    }
}
```



《PostgreSQL 数据库内核解读系列》课程资料分享



漠雪数据库分享

个人简介

于巍（花名漠雪）13年数据库内核开发和架构设计经验，从无到有参与阿里云PolarDB、华为GaussDB等多款数据库系列产品的内核架构设计和开发，作为数据库工具、安全首席架构，从无到有主导多款数据库运维工具（性能、故障、IDE、备份、安全）架构设计，多年美国、以色列等海外研发经验。目前主要负责横向拉通阿里云数据库各产品通用技术和架构演进，标准化，开源技术等工作。

一、PostgreSQL 内核解读系列课程

面向DBA、高校学生、内核爱好者，介绍PG内核架构、各模块基本原理、用法、代码实现。希望通过课程学习，让没有内核经验的同学和DBA，也可以进行简单的特性开发，更深入理解PG配置和运行原理。本课程包含15个章节，系统化介绍PG架构、SQL引擎、执行引擎、存储引擎、并发控制、HA、分布式等核心技术原理和代码实现。

- 1、PG数据库内核解读 - 第一节 系统概述 [视频链接](#) [演讲材料](#) [相关命令](#)
- 2、PG数据库内核解读 - 第二节 体系架构 [视频链接](#) [演讲材料](#) [相关命令](#)
- 3、PG数据库内核解读 - 第三节 存储管理1 [视频链接](#) [演讲材料](#)
- 4、PG数据库内核解读 - 第四节 存储管理2 [视频链接](#) [演讲材料](#)
- 5、PG数据库内核解读 - 第五节 索引1 [视频链接](#) [演讲材料](#) [相关命令](#)

二、数据库架构、技术趋势、运维解读

数据库架构、技术趋势、运维、性能、安全等分享

- 1、数据库在云原生时代的架构演进@CSDN在线峰会—新数据库时代 [视频链接](#) [演讲材料](#)
- 2、阿里云数据库性能测试及优化实践@信通院性能工具发布会 [视频链接](#) [演讲材料](#)
- 3、加密技术及阿里云数据库加密实践@信通院数安Talk [视频链接](#) [演讲材料](#)
- 4、数据库架构概述和技术演进趋势@专利代理师培训 [视频链接](#) [演讲材料](#)

三、国际主流数据库架构解读

国际主流OLTP、OLAP、NOSQL数据库架构、原理分享。待更新，[视频链接](#)

四、程序员的海外经历和趣闻

视频分享我在2015~2018年外派美国、以色列期间的趣闻轶事，展示和讲解拍摄的大量图片、视频及个人感受。待更新，[视频链接](#)

- 1、主要涉及地区: 美国（旧金山、圣何塞、洛杉矶、拉斯维加斯、圣地亚哥、芝加哥、萨克拉门托等）、以色列、加拿大、约旦、土耳其、印度等
- 2、主要涉及经历: 人文、地理、旅游、硅谷公司（Oracle、Google、Apple、AWS、IBM、Linkedin、Twitter等）、Startup公司、硅谷码工、高校（斯坦福、Beckley、UCLA、UIUC、芝加哥大学、以色列希伯来大学）、博物馆（洛杉矶盖蒂博物馆、电脑博物馆、火车博物馆、斯坦福博物馆、以色列艺术博物馆等）、美国以色列各城市租房和买房价格及流程、华人、Meetup（PG社区、教会、排球、Hiking、汽车、天文）、美食等。

CAICT 中国信通院

DBL 数据库应用创新实验室

数据库运维系列沙龙

——聊聊数据库的可观测性

日期DATE

2022.08.19

时间TIME

14:00-16:30 全程直播（云大说+TC601）

随着数据库的数据量和节点规模的日益增长，数据库运维管理的难度不断增大。数据库的可观测性愈发重要。数据库的可观测性不仅能帮助开发和运维人员更好地进行性能调优、故障诊断、容量和性能预测，同时也是数据库自治的重要基石。

— 会议议程 —

主持人：刘思源
中国信通院云计算与大数据研究所
大数据与区块链部业务主管

《openGauss可观测架构》
黄凯耀
华为技术有限公司
openGauss开源数据库首席架构师

《数据库可观测性能力的探讨》
徐 载（白鳍）
南京基石数据技术有限公司
CTO

《中国信通院数据库运维系列标准解读》
马嘉慧
中国信通院云计算与大数据研究所
大数据与区块链部工程师

《阿里云一站式数据库管理平台应对运维挑战》
于 巍（花名漠雪）
阿里云技术有限公司
数据库内核架构师

《跨越可观测性的鸿沟》
李 粒
平凯星辰（北京）科技有限公司
数据库高级产品经理

扫码观看直播

https://github.com/moxuedb/blog

